

Petko (Pete) Pavlovski

Senior Rust engineer — indexers, languages, P2P protocols, and the tooling around them.

\$ pavlovskipetko@gmail.com

cargopete.com

github.com/cargopete

linkedin.com/in/pete-pavlovski-07486a156

Sofia, Bulgaria · remote since 2019

I write the part that has to stay up. Most of it is still running. Seven years and eight months shipping in production, Rust for the last five. Accountant before that, which is where the habit of checking the number twice comes from.

	Six roles, one thread. Blockchain data infrastructure the whole way through, once the first two years had got the web work out of my system.	
Apr 2025 present	Fathom Senior Software Engineer – Rust backend systems for an AI-driven data platform. – Multi-workspace monorepo: services, storage, control plane. – Dataops workflows, and refactoring for performance and for the next person to read.	Remote, UAE
Jul 2022 Apr 2025	The Graph Senior Rust Engineer · GraphOps – At GraphOps, one of The Graph's core developer teams. – Designed and built Graphcast, a P2P messaging protocol on Waku and libp2p. – Backend services carrying the traffic, and a yew.rs frontend for the tooling. – Ran the demos and workshops, which is how you find out what the protocol is missing.	Remote
Dec 2021 Jul 2022	Kraken Rust Engineer – Developer tooling for testing exchange HTTP APIs.	Remote
Mar 2021 Dec 2021	The Graph Rust Engineer · LimeChain – At LimeChain, a dev shop, on a Graph Foundation grant. – Created Matchstick, the ecosystem's subgraph testing framework. Rust and WebAssembly.	Remote
Jan 2020 Mar 2021	OVO Energy Full Stack Engineer – Scheduling product for field engineer appointments. Kotlin and Quarkus microservices. – Built the Flutter app from scratch, and the React web client beside it.	Remote, UK
Jan 2019 Jan 2020	WeiChain Junior Software Engineer – Smart contracts on Ethereum and Aeternity. – Shipped a decentralised exchange to mainnet, plus an atomic swap widget.	Remote, Sofia
7y 8m	Seven years and eight months in production, Rust for the last five.	

Building now, and shipped and still running.

nuthatch — be your own indexer. One Rust binary, one command, a live indexed API in under two minutes. Follows the chain, seals segments to Parquet, serves SQL and MCP. No mandatory third-party data dependency. nuthatch-indexer.com

redstart — a language for authoring subgraphs. Schema, manifest and mappings collapse into one typed source that transpiles to AssemblyScript the canonical toolchain builds unmodified. If it compiles, it works. redstart-lang.com

yatr — a single-binary task runner whose cache is content-addressed and signed, so a shared cache entry can be trusted rather than hoped about. Speaks REAPI.

yetanothertaskrunner.com

matchstick — unit testing for subgraphs, in Rust and WebAssembly. The ecosystem standardised on it. 218 stars, 17 forks, still maintained by others. LimeChain for The Graph, on a Foundation grant, 2021

graphcast and **subgraph-radio** — a P2P messaging protocol for The Graph's indexer network, built on Waku and libp2p. Gossip, fault-tolerant delivery, and a radio pattern that lets anyone add a message type without touching the core. GraphOps, 2022–2025

The Night's Watch — the unglamorous half of The Graph: indexers, gateways, data services, judges and doctors. Lodestar, Dispatch, Foghorn, horizon-doctor, Graphite, learn-the-graph.com. 96 public repositories. github.com/nightswatchhq

What I do, and what I don't.

In scope

- Rust backends, services and CLIs
- Distributed systems and P2P protocol work
- Blockchain indexing and data infrastructure
- Developer tooling, languages and testing frameworks
- Taking something that half works and making it hold
- Remote, EET, overlapping most of a European day

Out of scope

- Frontend as the main job. I ship React and Flutter when a project needs it, but it is not what you want me for
- Smart contract auditing. I have written contracts, not audited them for a living
- ML research. I build the pipelines, not the models
- Relocation. Overlapping some US hours is fine
- Work where the architecture is settled and not open to a question